TRIE-AWARE TOOL MEMORY · FYP RESEARCH REPORT

Task B & Task E

Exact prefix-cache verification, the prefill measurement floor, tool-ordering effects on real cache hits, and trie-model calibration.

Measured on GPU · Function-call quality not yet evaluated

10.6×	4	38.15%	4.41%	1.2–1.5×
TTFT GAIN AT 200 TOOLS	CROSSOVER MENU SIZE	MEASURED REUSE, ALPHABETICAL	MEASURED REUSE, FREQUENCY	MODEL UNDER-PREDICTION

CONTENTS

Executive answer

All five Task B checks pass. Prefix reuse buys no TTFT at a 303-token prompt, but that is a measurement floor: padding tool menus to deployment-realistic sizes yields a 10.6× TTFT reduction at 200 tools, with a crossover near 4 tools. Validating the analytical trie estimate against measured cache hits inverts the earlier ordering recommendation — on shared catalogs, ordering by benchmark support front-loads task-specific tools and destroys the common prefix, making it nearly 9× worse than a stable global order.

This document answers Task B and Task E from `initial-research-brief.md`, and records two methodology corrections that changed earlier Task D conclusions. Every latency and cache figure here is a measured vLLM result, not an analytical estimate; where a number is extrapolated it is labelled as such.

Measurement environment

PROPERTY	VALUE
GPU	NVIDIA RTX 3090, 24 GiB (shared workstation, one GPU pinned per run)
Driver	580.173.02
Serving engine	vLLM 0.26.0, V1 engine
Model	`Qwen/Qwen3-0.6B`, bfloat16
KV block size	16 tokens
GPU KV blocks	11,807 (≈188,912 tokens)
Tool-call parsing	`--enable-auto-tool-choice --tool-call-parser hermes`
Decoding	`temperature=0`, `seed=0` (greedy)

Three environment prerequisites had to be discovered before vLLM would start at all. They are recorded because none are obvious and all three are fatal:

1. **Python 3.12 headers on `CPATH`.** Triton JIT-compiles a small `cuda_utils.c` and the system `python3.12-dev` package is not installed, so startup dies with `fatal error: Python.h: No such file or directory` whenever the torch compile cache misses. A dedicated `hdr312` conda environment supplies the headers.
2. **The venv `bin` on `PATH`.** FlashInfer shells out to `ninja`, which exists only inside the project venv; resolving the `vllm` executable by absolute path is not sufficient and startup fails with `FileNotFoundError: 'ninja'`.

3. `VLLM_USE_FLASHINFER_SAMPLER=0`. The system `nvcc` is CUDA 12.0 while torch is built for cu130, so FlashInfer's sampling kernel fails to compile against the wrong CUB (`BlockAdjacentDifference has no member`). Disabling the FlashInfer sampler is measurement-neutral here because all decoding is greedy.

Task B — Reproduce exact prefix caching

Question from the research brief

Set up a small instruct model with vLLM and verify that (1) two identical prompts reuse the prefix, (2) changing one tool definition causes a cache miss after the changed point, (3) changing the order of two tools changes the reusable prefix, (4) cache-enabled and cache-disabled generation produce the same result for the same token sequence, and (5) the experiment records prompt length, cached tokens, prefill latency, TTFT, and GPU memory.

Method and reasoning

A black-box probe (`scripts/vllm_prefix_cache_probe.py`) issues a fixed sequence of five scenarios against the OpenAI-compatible endpoint, diffing the vLLM Prometheus counters before and after each request. Three properties of the method matter more than the scenarios themselves.

Each trial starts genuinely cold. The probe calls `POST /reset_prefix_cache` before every trial, which requires the server to run with `VLLM_SERVER_DEV_MODE=1`. Without this only the first trial of a run is cold, and repeated trials silently measure a warm cache.

Every scenario runs five times with intervals. A single trial is not sufficient, and this is not a theoretical concern: an earlier single-trial run of this same experiment appeared to show a 3× TTFT gain (69 ms cold versus 23 ms warm). Repeating it showed per-trial cold TTFT of 93.9, 41.8, 42.7, 42.5, and 41.3 ms — the "speedup" was the server's first-ever request and nothing else. All intervals reported are 95% Student-t half-widths.

The served configuration is read back, not assumed. The probe reads `vllm:cache_config_info` and stores the actual `enable_prefix_caching` value with the results. This exists because the first attempt at check 4 was invalid: the control server was started by *omitting* `--enable-prefix-caching`, which does not disable the feature in vLLM V1. Both runs served `enable_prefix_caching=True`, so the comparison was a configuration against itself, and the identical cached-token counts were an artefact rather than evidence. `scripts/compare_probe_runs.py` now refuses to report output equality unless it can verify from the results file that the control served `enable_prefix_caching=False` and reported zero cached tokens.

KV-cache usage is sampled by a background thread *while each request is in flight*. `vllm:kv_cache_usage_perc` is an instantaneous gauge, so scraping it after a request returns always reads ~0 because the blocks have already been freed — which is why an earlier version of this experiment recorded no GPU memory evidence at all.

Task B results

Prompt length is 303 tokens except `changed_second_tool` at 309.

CHECK	CACHED / PROMPT TOKENS	REUSE	TTFT ON (MS)	TTFT OFF (MS)
Cold prompt	0 / 303	0.00%	52.4 ± 28.8	51.5 ± 27.3
Identical prompt reuse	288 / 303	95.05%	43.0 ± 1.2	40.5 ± 2.3
Changed second tool	128 / 309	41.42%	41.4 ± 1.1	42.3 ± 2.2
Reordered first two tools	48 / 303	15.84%	37.7 ± 5.7	43.0 ± 4.2
Original restored	288 / 303	95.05%	41.1 ± 2.1	39.4 ± 2.8

Peak KV-cache usage per request is 0.0019% of the 11,807-block cache, and mean inter-token latency is ~2.5 ms across all scenarios.

Task B answer

All five checks pass. Reuse behaves exactly as the exact-prefix model predicts: an identical prompt reuses 95% of it, editing one tool description cuts reuse to 41%, and reordering the first two tools cuts it to 16%. Cache-on and cache-off produce identical output text and tool calls in every scenario, against a control verified at `enable_prefix_caching=False` with zero cached tokens throughout.

At this prompt size, reuse bought no TTFT. No scenario separates cache-on from cache-off, and reuse fraction does not order TTFT — the 15.8%-reuse case has the lowest mean. A 303-token prompt prefills in roughly 40 ms end-to-end, dominated by fixed per-request overhead, so eliminating 288 tokens of prefill work is not resolvable above noise. This is a measurement floor, and Task E establishes where that floor is.

One limitation remains. The brief asks for the same result "for the same token sequence"; the comparison here is over response text and structured tool calls, because the OpenAI-compatible endpoint does not return token IDs. That is a weaker check than exact token equality.

Task E — First exact ToolTrie baseline

Question from the research brief

Implement a prompt-level prototype that receives a selected tool set, applies a deterministic order, serializes tools consistently, sends requests through vLLM with prefix caching enabled, and records exact prefix reuse and latency, without modifying KV tensors.

Method and reasoning

Menu construction. The benchmark workloads cannot test the hypothesis as they stand. ToolRet gold relevance sets and BFCL exposed menus both have a median of **one tool** per task (~124 and ~119 schema tokens). A one-item list cannot be reordered, and those prompt sizes are below the Task B measurement floor. Menus are therefore padded: each task keeps its full gold tool set, and distractors are added from a fixed global catalog up to a target size. This models a deployment that keeps the same connected tools loaded across requests, and it makes menu size a swept parameter.

The padding must come from **one global ranking**, not be resampled per task. An initial implementation seeded the distractor shuffle with the task's own gold tools, giving every task a unique menu; measured reuse collapsed to 1.23%. Since a deployment serves largely the same catalog to every request, padding is now drawn from a single seeded ranking, and different tasks receive nearly identical distractors. `tests/test_prompting.py` pins this property.

Prefill sweep. For each menu size, the same request is issued cold (after a prefix-cache reset) and then warm (an identical repeat), five times. The whole sweep is then repeated against a `--no-enable-prefix-caching` control server.

A note on inference: within-run cold-versus-warm separation is *not* sufficient evidence of a caching benefit. The control run — where no cache exists at all — also shows "separated" intervals at one tool. Only the cache-on versus cache-off comparison is trustworthy, and the crossover reported below uses that.

Estimate validation. `scripts/validate_reuse_estimate.py` replays a generated workload against vLLM and compares the analytical `trie_metrics` prediction with the summed `vllm:prompt_tokens_cached`. Before this, every reuse percentage in the project was an unvalidated model.

Task E results

Prefill sweep, one gold tool padded to the stated menu size, 5 trials per point:

MENU TOOLS	PROMPT TOKENS	COLD TTFT ON (MS)	WARM TTFT ON (MS)	WARM TTFT OFF (MS)	SPEEDUP
1	250	25.6	23.4	23.1	1.0×
4	433	25.2	16.9	23.7	1.4×
16	1,771	62.1	35.3	56.5	1.6×
64	6,742	248.8	54.1	245.6	4.5×
128	13,422	647.8	85.2	640.7	7.5×
200	20,627	1282.7	120.7	1275.2	10.6×

The cache-disabled control stays flat — cold within noise of warm at every size — so the gap is prefix caching and not a warmup artefact. The 1- and 4-tool points are noisy (4 tools shows warm 16.9 ms, below the 1-tool 23.4 ms, which is not physical) and should not be read closely.

Analytical estimate versus measured cache hits, 200 BFCL tasks at 64-tool menus:

WORKLOAD	PREDICTED CACHEABLE	MEASURED CACHED	MEASURED/PREDICTED	PREDICTED REUSE	MEASURED REUSE
alphabetical	428,016	526,432	1.23×	38.42%	38.15%
frequency	40,448	60,864	1.50×	3.63%	4.41%

Task E answer

Prefix caching is worth a great deal, once catalogs are realistic. The crossover is around 4 tools / 433 prompt tokens, and the benefit grows steeply: at 200 tools it is a 10.6× TTFT reduction. The Task B null result was a property of the prompt size, not of prefix caching.

The Task D ordering recommendation inverts on realistic menus. Task D recommended frequency ordering from an analytical model over gold-only menus. On padded menus it is nearly 9× worse than alphabetical (4.41% versus 38.15% measured reuse). The mechanism is clear: ordering by benchmark support ranks the *task-specific* gold tools first and pushes the shared catalog behind them, so the common prefix is destroyed. A stable global order that ignores task-specificity keeps the shared catalog in front. Frequency ordering only wins when the entire menu is task-specific, which is not what a connected tool catalog looks like.

The analytical trie model under-predicts, and is usable with a correction. Measured hits exceed predictions by 1.23× (alphabetical) and 1.50× (frequency). The model ignores the chat template and system preamble, which are identical across requests and are themselves cached, and that outweighs the partial blocks it loses at tool boundaries. Calibration is tighter where reuse is high, so the model is most trustworthy in the regime that matters.

Does the winning ordering preserve function-call quality?

Method. 100 BFCL tasks, stratified 20 per category (`simple_python`, `multiple`, `parallel`, `parallel_multiple`, `irrelevance`) rather than the first-200-in-file-order sample used elsewhere, which is entirely `irrelevance` tasks and cannot test name/argument accuracy at all. Each task's gold tools are padded to a 64-tool menu, matching the size already validated for cache reuse. One request per task, `temperature=0`, `seed=0`, against the same server used for the reuse validation. Predicted tool calls are scored against BFCL's `possible_answer` ground truth (`src/tatm/bfcl_score.py`): function-name accuracy, full accuracy (name and every argument correct, order-independent for parallel calls), and no-tool accuracy for the irrelevance category. This is a reduced reimplementation of the official Gorilla/BFCL checker, not a vendored copy — no code execution, no per-type coercion table, any predicted argument absent from ground truth counts as a mismatch.

One correction was needed before this produced usable data: Qwen3 emits a `<think>` reasoning block by default, which consumed the entire 128-token budget before any tool call was produced, giving 2.5% accuracy on *both* orderings identically — a generation-budget artefact, not a quality result. Fixed with `chat_template_kwargs: {"enable_thinking": false}` (`--disable-thinking` on `replay_vllm_workload.py`).

Results, 80 scorable relevance tasks + 20 irrelevance tasks per ordering:

ORDERING	FUNCTION-NAME ACCURACY	FULL ACCURACY (NAME + ARGS)	NO-TOOL ACCURACY
alphabetical	67.5%	47.5%	95.0%
frequency	77.5%	51.25%	85.0%

Answer: no, not cleanly — the ordering that wins on reuse is not the ordering that wins on quality. Frequency ordering, measured nearly 9× worse for cache reuse, scores *higher* on function selection (77.5% vs 67.5% name accuracy) and higher on full correctness, while alphabetical is markedly better at correctly declining when no tool applies (95.0% vs 85.0% no-tool accuracy). At n=80 the name-accuracy gap is within roughly one standard error of the sampling noise — not something to treat as settled — but the qualitative shape (frequency helps selection, alphabetical helps abstention) held in every category checked, and a free reuse win is not free of tradeoffs.

This changes the practical recommendation from Task E's answer above. "Alphabetical wins" was true only for cache reuse. A deployment that cares about tool-selection correctness cannot adopt alphabetical ordering on reuse grounds alone without accepting a plausible selection-accuracy cost, and cannot adopt frequency ordering without accepting both a 9× reuse loss and a higher false-call rate on irrelevant requests. Neither ordering dominates the other once quality is in the picture — this is exactly the tradeoff the brief's Q7 was checking for, and the reason no ordering could be recommended before this measurement existed.

Caveats: single run per ordering, no repeats or confidence intervals — unlike the latency tables above, these are point estimates from 100 requests each, not five-trial means. 20 tasks per category is small enough that domain-level splits should be read as directional, not conclusive. A larger, repeated run is needed before this becomes a number to design a system around.

Is TTFT optimizable?

Decomposing the sweep over its two clean high-size points (6,742 → 20,627 prompt tokens):

COMPONENT	MEASURED	OPTIMIZABLE
Fixed per-request overhead	~20 ms	No — scheduling, tokenization, HTTP, sampler setup
Marginal prefill, cold	~75 µs/token	Yes — this is what caching removes
Marginal cost, fully warm	~4.8 µs/token	No — residual at ~100% cache hit

Prefix caching eliminates roughly 93% of the per-token prefill cost, but not all of it: warm TTFT still climbs from 23 ms to 121 ms across the sweep even though 20,624 of 20,627 tokens were cache hits, because the first decode step attends over the full KV and block-table setup scales with context length.

The remaining lever is therefore the **reuse fraction**, not the per-token cost. Modelling TTFT as `floor + (1−r)·cold_slope·N + r·warm_slope·N` at 6,742 tokens gives ~175 ms at the measured r = 0.38 and ~74 ms at r = 0.90. This is an extrapolation: the validation run records cached tokens but not TTFT, so the partial-reuse latency has not been measured directly. Doing so is one short GPU run and should replace this model with data.

Task D correction — locality was not measurable

`trie_metrics` retained every node forever. Its output therefore depends only on the *multiset* of requests and not on their order, verified by 200 random reorderings producing identical metrics. Two consequences followed:

- `session_bursty` is a permutation of `empirical`, so it was **identical by construction** — which is exactly what the earlier report showed (31.35% and 36.52% in both rows for ToolRet).
- The `uniform` and `skewed` replays differ only because they resample *with replacement*, so the headline "skewed → 80.18% reuse" measured support concentration and duplicate requests, not locality.

Request order can only matter once capacity forces eviction. `bounded_trie_metrics` adds a finite cache with leaf-first LRU eviction — a node may only be evicted once it has no children, mirroring a radix tree that cannot free a block while a longer path depends on it. With capacity, the replays diverge.

The finding is not the expected one. ToolRet's natural file order is *more* local than the synthetic bursty replay (31.35% versus 27.10% reuse at 25% capacity), because its file order is already 99.86% same-domain adjacent. The synthetic replay, which shuffles groups and shuffles within them, is less local than the data already was.

Operational note: the reports are generated

`reports/*.md` are produced by `src/tatm/reporting.py` and overwritten on every `scripts/run_pipeline.py` run. Measured GPU results are read back from `cluster/results/` by `load_probe_results` and rendered into the findings report, so they survive regeneration. Hand-editing a generated report silently loses the edit on the next pipeline run.

What has not been measured

- **Function-call quality at scale.** A first pass exists (100 tasks, above) and shows the two orderings trade off differently on reuse versus quality — but it is one run with no repeats, small per-category samples, and only two of six orderings scored.
- **Four of six orderings.** Only alphabetical and frequency were validated against measured cache hits.
- Rendered full-prompt token IDs and exact block boundaries.
- TTFT on partial-reuse workloads (see the extrapolation above).
- GPU/KV memory under eviction pressure at large menu sizes.

Reproduction

```
# Data and local analysis
python scripts/download_datasets.py
python scripts/run_pipeline.py

# Serving experiments; see cluster/README.md for the required environment
python scripts/vllm_prefix_cache_probe.py --run-label cache-enabled --repeats 5 \
  --output cluster/results/cache-enabled.json
python scripts/prefill_sweep.py --run-label sweep-cache-enabled --repeats 5 \
  --output cluster/results/prefill-sweep-enabled.json
python scripts/build_cluster_workload.py --partition bfcl --ordering alphabetical \
  --menu-size 64 --limit 200 --output data/processed/bfcl-alpha-menu64.jsonl
python scripts/validate_reuse_estimate.py \
  --input data/processed/bfcl-alpha-menu64.jsonl \
  --run-label validate-menu64-alphabetical \
  --output cluster/results/validate-menu64-alphabetical.json

# Function-call quality under ordering
python scripts/build_bfcl_quality_workload.py --ordering alphabetical \
  --per-domain 20 --menu-size 64 \
  --output data/processed/bfcl-quality-alphabetical.jsonl
python scripts/replay_vllm_workload.py \
  --input data/processed/bfcl-quality-alphabetical.jsonl \
  --run-label bfcl-quality-alphabetical --max-tokens 128 --disable-thinking \
  --output cluster/results/bfcl-quality-alphabetical.json
python scripts/score_bfcl_quality.py \
  --replay-result cluster/results/bfcl-quality-alphabetical.json \
  --output cluster/results/bfcl-quality-alphabetical-score.json
# repeat with --ordering frequency for the comparison row

python -m pytest tests/ -q
```

Raw measurement outputs are written to `cluster/results/`, which is gitignored; the figures in this document are reproduced from those files.